Keywords: flow matching • language generation • diffusion language models • fast inference

Rectified Language Flow

Straight-Path Flow Matching in Token Embedding Space Beats Autoregressive Baselines at 15–19× Inference Speedup

By Baiyu Su, Runlong Liao, Lizhang Chen, Qiang Liu

arXiv:2610.33811 • cs.AI / cs.LG • University of Texas at Austin

Rectified Language Flow (ReLF) generates text by training a flow model to move noise continuously toward token embeddings along straight (rectified) paths, enabling accurate generation in as few as one to four function evaluations. At batch size 1 it produces 1,024-token sequences 15–19× faster than a matched autoregressive baseline, while achieving lower generative perplexity on OpenWebText and outperforming all prior continuous-flow language models.

AT A GLANCE

Problem: Autoregressive models are slow at inference due to serial token generation; discrete diffusion models require many costly denoising steps over a large discrete space.

Why it matters: Fast, high-quality text generation is critical for interactive AI, serving efficiency, and real-time applications such as code completion and dialogue.

Method: Continuous flow matching in Hellinger-SVD token embeddings with self-conditioning guidance, regularized reflow for distillation, and singular logits for discrete commitment.

Result: Lower perplexity than autoregressive baselines on OpenWebText; 15–19× faster generation at batch size 1 with 4 function evaluations.

Evidence: OpenWebText language modeling benchmarks across three model sizes; ablation over number of function evaluations.

The Big Picture

Large language models have converged on autoregressive (AR) generation: at each step the model conditions on all previous tokens and samples the next one. This left-to-right serial process is the primary bottleneck at inference time—each token requires a full forward pass, and a sequence of T tokens requires T passes.

Diffusion language models offer an alternative: start from a noisy sequence and iteratively refine it over K denoising steps. Because denoising steps can be parallelized across positions, the number of forward passes is $K \ll T$ for long sequences. But discrete diffusion models operate over a large categorical space at each position, introducing its own inefficiencies, and K is typically still in the tens to hundreds.

ReLF takes a third approach: flow matching in a continuous embedding space. A flow model is trained to transport a simple noise distribution p_0 (e.g., Gaussian) to the distribution of token embeddings p_1. The rectified flow objective enforces straight paths between noise and data, enabling accurate generation with as few as 1–4 function evaluations.

Why Existing Approaches Fall Short

Autoregressive models require T serial forward passes for a length-T sequence. Speculative decoding and other parallelization heuristics reduce this in practice but don't change the fundamental asymptotic cost.

Discrete diffusion models work directly in token space, avoiding the need to commit to embeddings. However, the discrete state space is large (vocabulary $\sim$ 50k), and the denoising objective must define meaningful noise over discrete categories. Prior models report 32–64 steps to achieve quality competitive with AR models.

Prior continuous-flow language models (ELF, DiLaDiff) use learned continuous embeddings that are unstable to train and require reconstruction losses that decouple the flow from the discrete output space. Committing a continuous embedding to a discrete token at generation time via nearest-neighbor lookup introduces discretization error that accumulates over the sequence.

Core Mechanism

ReLF's four key components work together to address these issues:

1. Hellinger-SVD Embeddings. Token embeddings are initialized from the SVD of the Hellinger-transformed co-occurrence matrix of the training corpus. This produces a geometry in which frequent co-occurring tokens are nearby, providing a meaningful flow target that is fixed throughout training. Crucially, no embedding training is needed.

2. Self-Conditioning Guidance. At each training step the model is first run with the corrupted input x_t to produce a preliminary estimate $\hat{x}_0$. On the same forward pass (with gradients detached), $\hat{x}_0$ is concatenated to x_t and passed to the model again. A cross-entropy loss between $\hat{x}_0$ and the true x_0 trains the guidance branch. This conditioning helps the model resolve ambiguous intermediate states near $t = 0.5$.

3. Regularized Reflow. After initial training, a reflow stage distills the learned curved flow into a straighter one by training on (x_0, x_1) pairs sampled from the original model's trajectories. A regularization term penalizes degenerate straight lines that pass outside the data manifold, preventing the reflow from creating diversity collapse.

4. Singular Logits. As $t \to 1$ the flow approaches the data manifold and must commit to a discrete token. Singular logits is a differentiable mechanism that computes token logits from the flow's velocity direction at $t = 1$, rather than from the embedding value itself, enabling sharp, gradient-compatible token selection.

Technical Formulation

Let $x_0 \sim \mathcal{N}(0, I)$ be noise and $x_1 \sim p_{\text{data}}$ be a token embedding sequence. The rectified flow objective minimizes:

$$\mathcal{L}_{\text{flow}} = \mathbb{E}_{t,x_0,x_1}\left[\|v_\theta(x_t, t) - (x_1 - x_0)\|^2\right]$$

where $x_t = (1-t)x_0 + t\,x_1$ and v_θ is the learned velocity field.

The self-conditioning guidance loss augments training with:

$$\mathcal{L}_{\text{sc}} = -\log p_\theta(x_1 \mid \hat{x}_0, x_t, t)$$

where $\hat{x}_0 = x_t + (1-t)\,v_\theta(x_t, t)$ is the preliminary clean estimate.

The regularized reflow loss is:

$$\mathcal{L}_{\text{reflow}} = \mathcal{L}_{\text{flow}}^{\text{straight}} + \lambda \cdot \mathbb{E}\big[\text{dist}(x_t^{\text{reflow}}, \mathcal{M})\big]$$

where $\mathcal{M}$ is an approximation of the data manifold and λ is a regularization weight. Singular logits are defined as:

$$\ell_v = W_{\text{vocab}}\, v_\theta(x_1, 1) \cdot \|W_{\text{vocab}}\|^{-1}$$

mapping the terminal velocity to logits via the vocabulary matrix.

Learning or Inference Procedure

Training:

1. Pre-compute Hellinger-SVD embeddings from corpus statistics.
2. Train v_θ with $\mathcal{L}_{\text{flow}} + \alpha \cdot \mathcal{L}_{\text{sc}}$ for N steps.
3. Collect (x_0, x_1) pairs from teacher model trajectories.
4. Fine-tune with $\mathcal{L}_{\text{reflow}}$ for $M \ll N$ additional steps.

Inference: Sample $x_0 \sim \mathcal{N}(0, I)$, integrate the velocity field over $K \in \{1, 2, 4, 8, 16, 64\}$ Euler steps, apply singular logits at $t = 1$ to decode token distributions, and sample.

What the Guarantee Says

The rectified flow objective guarantees that the expected trajectory length $\mathbb{E}[\int_0^1 \|v_\theta\|\,dt]$ is minimized when trajectories are straight, which corresponds to the lower bound on the number of function evaluations required for a given discretization error. While this is an asymptotic argument, the regularized reflow empirically achieves near-straight paths and the 1-step sample quality confirms the practical benefit. No PAC-style generalization bound is provided; the guarantee is about transport efficiency, not sample quality.

Experimental Findings

- **Perplexity:** On OpenWebText, with 64 function evaluations, ReLF achieves **lower perplexity than AR baselines** at matched parameter counts for all three model sizes (90M, 350M, 1.2B).
- **Speed:** At 4 function evaluations, ReLF generates 1,024-token sequences **15–19× faster** than the AR baseline at batch size 1, with less than 0.5 perplexity units of degradation relative to the 64-step setting.
- **Comparison to ELF:** ReLF outperforms ELF at all model sizes by 1.2–2.8 perplexity points.
- **Diversity:** At matched perplexity, ReLF self-BLEU scores are lower (more diverse) than the AR baseline, suggesting the flow's sampling process does not collapse to near-deterministic outputs.

Ablations and Interpretation

Hellinger-SVD vs. learned embeddings: Learned embeddings reduce final perplexity by 0.3 points on average but add training instability; the fixed Hellinger-SVD embeddings are more robust and are used in all reported experiments.

Without self-conditioning: Removing self-conditioning increases perplexity by 1.8–3.4 points, with the largest degradation at small K (few steps). This confirms that guidance is most critical in the low-step regime.

Without reflow: Without the reflow stage, 4-step generation degrades by 5.1 perplexity points relative to the 64-step setting. With reflow, the gap closes to < 0.5 points.

Singular logits vs. nearest neighbor: Replacing singular logits with nearest-neighbor lookup in embedding space increases perplexity by 0.7–1.1 points and degrades diversity. The directional signal at $t = 1$ is more discriminative than the position signal.

Reference

Baiyu Su, Runlong Liao, Lizhang Chen, Qiang Liu. **Rectified Language Flow.** arXiv:2610.33811. 2026. https://arxiv.org/abs/2610.33811

Keywords: vision-and-language navigation • hierarchical reinforcement learning • embodied AI • continuous environments

MATCH: Execution-Aware Hierarchical Reinforcement Learning for Vision-and-Language Navigation

Joint End-to-End Training of Planner and Executor Sets New State-of-the-Art on R2R-CE and RxR-CE at 0.5B Parameters

By Shuhao Ye, Yufei Wei, Demao Wang, Shichao Zhai, Rong Xiong, Yue Wang

arXiv:2610.38248 • cs.RO • submitted 28 Oct 2026

Hierarchical VLN agents that optimize the high-level planner while keeping the low-level executor fixed suffer from execution blindness: the planner proposes waypoints without knowing whether the executor can reach them. MATCH closes this loop by jointly training a language-conditioned planner and a goal-conditioned executor end-to-end with reinforcement learning, routing execution feedback back to inform future waypoint selection. With only 0.5B parameters, MATCH sets new state-of-the-art on R2R-CE (62.4% SR) and RxR-CE (55.8% SR).

AT A GLANCE

Problem: In VLN-CE, hierarchical planners propose waypoints without knowing what the executor can achieve, causing cascading failures.

Why it matters: Embodied navigation in continuous 3D environments with language instructions is a core capability for household robots and assistive AI.

Method: End-to-end RL training of planner and executor; execution-status feedback from executor to planner at each step; shared visual backbone.

Result: 62.4% SR on R2R-CE (val unseen) and 55.8% SR on RxR-CE (val unseen), surpassing all prior methods.

Evidence: R2R-CE and RxR-CE benchmarks, val unseen and test splits; ablations over execution feedback and joint training.

The Big Picture

Vision-and-Language Navigation (VLN) tests an agent's ability to follow free-form instructions like "walk past the sofa, turn right at the kitchen, and stop in front of the bookshelf" in a photo-realistic 3D environment. In Continuous Environments (VLN-CE), the agent must produce raw motor actions—translation velocity and rotation—rather than selecting named waypoints on a graph, making the problem significantly harder.

Hierarchical approaches decompose VLN-CE into two sub-problems: a *planner* that parses the instruction and generates a sequence of subgoals (waypoints), and an *executor* that drives the robot from its current position to each waypoint. This decomposition is intuitive but introduces a structural mismatch: the planner has no direct signal about whether its proposed waypoints are achievable given the current room geometry and the executor's capabilities.

Why Existing Approaches Fall Short

Fixed executor, trained planner: Most prior hierarchical VLN-CE systems pretrain or fix the low-level executor (typically a model-predictive controller or a simple goal-conditioned policy) and focus all learning effort on the high-level planner. The planner therefore cannot receive gradient signal from execution failures and learns only from its own prediction errors.

Mismatched subgoal abstraction: Even when both components are trained, they are typically trained separately on different objectives (language-goal matching for the planner; reaching accuracy for the executor), meaning there is no shared objective that penalizes the planner for proposing unreachable waypoints.

No execution feedback: The planner selects its next waypoint based on the instruction and the current visual observation, but not on how the last waypoint was executed. Information about partial failures (executor reached 70% of the waypoint but collided) is discarded between planning steps.

Core Mechanism

Language-Conditioned Planner. Given the full instruction I and the current visual observation o_t, the planner $\pi_P(w_t \mid I, o_t, h_t^P)$ produces a waypoint w_t as a displacement vector relative to the current agent pose, where h_t^P is the planner's recurrent state.

Goal-Conditioned Executor. Given the current observation o_t^{low} and the target waypoint w_t, the executor $\pi_E(a_t \mid o_t^{\text{low}}, w_t, h_t^E)$ produces a primitive action a_t (linear velocity, angular velocity).

Execution-Status Feedback. After the executor attempts waypoint w_t, the achieved displacement and an obstacle-collision indicator are compressed into an execution-status vector s_t and appended to the planner's observation at step $t+1$: $h_{t+1}^P = \text{RNN}(h_t^P, [o_{t+1}, s_t])$.

Joint RL Training. The planner and executor share a single reward: the navigation success reward at the

episode terminus, discounted back through the waypoint sequence. Both are updated by PPO in a shared replay buffer.

Technical Formulation

The planner's policy is:

$$w_t = \pi_P\big(I,\, o_t,\, h_t^P,\, s_{t-1}\big)$$

The executor's policy unrolls for T_{exec} steps per waypoint:

$$a_\tau = \pi_E\big(o_\tau^{\text{low}},\, w_t - p_\tau,\, h_\tau^E\big)$$

where p_τ is the agent's current position.

The reward shaping for the planner uses a distance-based dense reward:

$$r_t^P = d(p_t, g) - d(p_{t+1}, g) - \lambda_c \cdot \mathbf{1}[\text{collision}_t]$$

where g is the goal location and λ_c penalizes collisions. The executor receives a waypoint-reaching reward:

$$r_\tau^E = \max(0,\; d(p_\tau, w_t) - d(p_{\tau+1}, w_t))$$

The full PPO objective optimizes π_P and π_E jointly:

$$\mathcal{L} = \mathcal{L}_{\text{PPO}}(\pi_P) + \beta\, \mathcal{L}_{\text{PPO}}(\pi_E)$$

Learning or Inference Procedure

Pretraining. The shared visual backbone (ViT-B) is initialized from a pretrained visual-semantic embedding. The instruction encoder is initialized from a compact BERT variant.

Stage 1 – Imitation Learning. Both planner and executor are pretrained with behavior cloning on expert trajectories for 10k steps.

Stage 2 – Joint RL Fine-tuning. PPO is applied for 50k environment steps. The execution-feedback vector is activated after Stage 1, ensuring the planner does not receive noisy feedback from an untrained executor.

Inference. The planner generates one waypoint at a time, the executor runs for up to $T_{\text{exec}} = 20$ steps, and the planner then receives the execution-status feedback and produces the next waypoint.

What the Guarantee Says

No theoretical guarantee is claimed. The paper's argument is empirical: execution-aware training produces better navigation policies, and the ablation studies isolate which components contribute. The success-rate metric on val unseen splits, which feature environments not seen during training, provides evidence of generalization.

Experimental Findings

- **R2R-CE val unseen:** MATCH achieves **SR = 62.4%**, **GP = 65.1%**, **NE = 3.8 m**, surpassing the prior SOTA (SR = 58.1%) by 4.3 points.
- **RxR-CE val unseen:** MATCH achieves **SR = 55.8%**, improving over the prior best (51.3%) by 4.5 points.
- **Parameter efficiency:** MATCH achieves these results with **0.5B parameters**, outperforming prior methods that use 2–7B parameters.
- **Collision rate:** Joint RL training with the collision penalty reduces the executor's collision rate by 31% relative to the imitation-only baseline.

Ablations and Interpretation

Execution feedback removed: Removing the execution-status vector and training jointly on reward alone reduces SR on R2R-CE by 3.1 points, confirming that the information channel—not just joint training—is critical.

Fixed executor (standard hierarchical baseline): Fixing the executor at its imitation-learning checkpoint and training only the planner reduces SR by 5.7 points, establishing the cost of decoupled training.

Imitation only (no RL): Without RL, SR is 54.6%, 7.8 points below MATCH. RL is necessary to generalize to execution scenarios not covered by expert data.

Waypoint granularity: Decreasing the maximum number of waypoints per episode degrades SR smoothly, suggesting the planner learns to use waypoints efficiently rather than uniformly spacing them.

Reference

Shuhao Ye, Yufei Wei, Demao Wang, Shichao Zhai, Rong Xiong, Yue Wang. **MATCH: Execution-Aware Hierarchical Reinforcement Learning for Vision-and-Language Navigation in Continuous Environment.** arXiv:2610.38248. 2026. https://arxiv.org/abs/2610.38248

Keywords: scientific discovery • LLM agents • physical laws • autonomous exploration

Can LLMs Uncover Hidden Physical Laws through Free Exploration?

LongChang Benchmarks Open-Ended Physical Discovery in Altered-Physics MuJoCo Worlds: 11 of 12 Frontier Models Score Zero

By Zhen Yang, Wen Wang, Xander Xu, Ying-Cong Chen

arXiv:2610.37649 • cs.CL • NeurIPS 2026 ML4PS Workshop

Eleven of twelve frontier large language models fail entirely at open-ended physical discovery: placed in a MuJoCo simulation with silently altered physical laws and given tools to run experiments freely, they detect anomalies but cannot formulate the governing equation. Only GPT-6-Astra succeeds at all, with a 16.7% discovery rate. The result documents a sharp gap between competent reasoning within known frameworks and genuine hypothesis-driven scientific inquiry.

AT A GLANCE

Problem: It is unknown whether LLMs can perform genuine open-ended scientific discovery—designing experiments, iterating hypotheses, and formulating laws without being prompted.

Why it matters: Autonomous AI scientists could accelerate research, but only if their discovery capabilities go beyond pattern matching on known science.

Method: LongChang framework: MuJoCo worlds with silently altered physics; LLM agents given Python tools for free exploration; discovery scored by law-formulation accuracy.

Result: 11 of 12 frontier models score 0%; GPT-6-Astra achieves 16.7% (1 of 6 trials).

Evidence: 12 frontier LLMs evaluated across 6 altered-physics environments; qualitative analysis of exploration patterns.

The Big Picture

Scientific discovery requires more than retrieval or deduction from known facts. It requires *abduction*: noticing that the world behaves unexpectedly, designing experiments to distinguish candidate explanations, accumulating evidence, and arriving at a formal law. This process is epistemically distinct from the question-answering and reasoning tasks on which LLMs currently excel.

The question addressed by this paper is sharply defined: given a simulation world in which one or more physical constants have been silently altered, and given a set of Python tools with which to run experiments, can an LLM figure out which law has changed and state it correctly? No hint is given about which quantity to vary. No labeled data is provided. The agent must decide what to measure, how to measure it, and what to conclude.

Why Existing Approaches Fall Short

Equation discovery benchmarks (e.g., SRBench, AI Feynman) provide labeled (x_i, y_i) pairs and ask models to fit a symbolic expression. This is regression, not discovery: the model is told which variables are relevant and is given cleaned, ready-made data.

Scientific reasoning benchmarks (e.g., SciBench) test whether LLMs can answer physics questions correctly. This requires application of known laws, not their formulation.

Agent-with-tool benchmarks (e.g., WebArena, SciAgent) give agents access to search or computation tools but define the task question explicitly. The agent does not decide what question to ask.

LongChang differs on all three dimensions: no labeled data, no known applicable law, no predefined question.

Core Mechanism

Altered-Physics Worlds. A MuJoCo simulation is configured with one or more modified constants: gravitational acceleration g, coefficient of restitution e, or air drag coefficient k. The alteration is sampled from a set of plausible values (e.g., $g \in \{4, 6, 14, 18\}\,\mathrm{m/s^2}$) to prevent trivial detection.

Tool Suite. The agent is provided with Python tools to:

- spawn and reset objects (balls, ramps, pendulums),
- apply forces or initial velocities,
- query sensor readings (position, velocity, force) over time,
- log numerical results to a file.

The agent writes and executes Python code calling these tools.

Exploration Protocol. The agent receives a budget of N tool calls (typically $N = 200$). It is told only that "some physical properties of this world may differ from the standard physics you know." It must determine what those properties are and state them formally by the end of its budget.

Scoring. A discovery is marked successful if the agent's final stated law is numerically correct within $\pm 5\%$ of the true altered value, and identifies the correct physical quantity. Partial credit is given for correct anomaly

identification alone.

Technical Formulation

Let $\mathcal{W}_c$ denote a world with altered physical constant $c \in \mathcal{C}$. The agent is a language model $\mathcal{A}$ that generates a sequence of tool calls $a_1, a_2, \ldots, a_N$ and a terminal law statement $\hat{L}$.

The discovery score is:

$$S(\mathcal{A}, \mathcal{W}_c) = \mathbf{1}[|\hat{c} - c|/c < \epsilon \ \wedge \ \hat{q} = q]$$

where $\hat{c}$ is the agent's estimated value, q is the correct quantity, and $\epsilon = 0.05$.

The Collective Anomaly Detection score (CAD) measures partial credit:

$$\text{CAD}(\mathcal{A}) = \mathbb{E}_{\mathcal{W}_c}[\mathbf{1}[\text{agent flags anomaly in quantity } q]]$$

The gap between CAD and S quantifies the failure mode: detecting that something is wrong but not determining what.

Learning or Inference Procedure

No training is performed; all models are evaluated zero-shot with their standard system prompts. The evaluation is therefore a direct test of models' out-of-the-box capability for autonomous scientific discovery. Each model is given a standardized initial message describing the tools and the open-ended task.

Chain-of-thought prompting is used (the model is asked to reason before each tool call), but no few-shot examples of discovery are provided to avoid biasing the exploration strategy.

What the Guarantee Says

No formal guarantee is claimed. The paper's contribution is empirical: it establishes a lower bound of ≈0% on the discovery ability of current frontier models (with one exception), and provides a repeatable benchmark for tracking progress. The LongChang framework's automatic world generation ensures that future evaluations can use fresh worlds not seen in any training data, preventing benchmark saturation.

Experimental Findings

- **Discovery rates:** GPT-6-Astra: **16.7%**; all other 11 models: **0%**.
- **Anomaly detection (CAD):** GPT-6-Astra: 66.7%; average across all models: 41.2%. Models can detect that something is wrong far more reliably than they can determine what is wrong.
- **Hypothesis revision:** All models converge on an initial hypothesis after the first 20–50 tool calls and rarely revise it, even when subsequent measurements contradict it.
- **Tool use quality:** Models tend to run many single-variable experiments but fail to design the controlled comparisons needed to isolate the altered constant.
- **Gravity vs. drag:** Altered gravity is discovered more often than altered drag (GPT-6-Astra discovers gravity in all successful trials), consistent with gravity's more direct effect on commonly simulated trajectories.

Ablations and Interpretation

Provided anomaly hint: When the agent is told "the gravitational acceleration in this world has been altered," success rates rise to 50%+ for GPT-6-Astra and 16.7% for several other models. The bottleneck is therefore anomaly targeting, not law formulation.

Reduced tool budget ($N = 50$): GPT-6-Astra's discovery rate drops from 16.7% to 0%; it needs ∼150 calls to triangulate the correct value. Other models are equally unsuccessful at $N = 50$.

Single-alteration vs. multi-alteration worlds: Success rates drop to 0% for all models when two constants are simultaneously altered, suggesting that current LLMs cannot disentangle compound perturbations.

Failure mode categorization: The authors classify failures as: (1) no anomaly detected (17%), (2) anomaly detected but wrong quantity identified (51%), and (3) correct quantity but incorrect value (32%). Type (3) is encouraging for the future: the bottleneck is precise estimation, not total blindness.

Reference

Zhen Yang, Wen Wang, Xander Xu, Ying-Cong Chen. **Can LLMs Uncover Hidden Physical Laws through Free Exploration?** arXiv:2610.37649. NeurIPS 2026 ML4PS Workshop. 2026. https://arxiv.org/abs/2610.37649

Keywords: model-based reinforcement learning • vision-language models • latent world models • V-JEPA

Vision-Language Agents Can Learn in Their Own Latent Worlds

Dwell Replaces Pixel-Space World Models with Recurrent Latent Transition in Frozen V-JEPA2 Embeddings, Cutting Imagination Compute by 2.3×

By Haoxi Li, Sikai Bai, Jianfei Ma, Qinglin Hou, Xiaocheng Lu, Tao Han, Jie Zhang, Song Guo

arXiv:2610.36925 • cs.AI • submitted 27 Oct 2026

Dwell builds a world model for vision-language agents that predicts the next latent state directly in the frozen embedding space of V-JEPA2, rather than reconstructing future frames. This removes the costly pixel decoder from model-based RL for VLMs, reducing imagination compute by 2.3× while achieving the strongest overall performance among trainable methods across five interactive visual environments.

AT A GLANCE

Problem: Pixel-space world models for VLM agents are computationally expensive and may focus on perceptually salient but task-irrelevant details.

Why it matters: Sample-efficient VLM agents are needed for tasks where environment interaction is costly (robotics, real-world planning, interactive apps).

Method: Recurrent latent transition model in frozen V-JEPA2 embedding space; VLM imagines ahead in latent space; no pixel decoder.

Result: Best overall performance across 5 visual environments among trainable methods; 2.3× lower imagination compute.

Evidence: 5 interactive visual environments; ablation over world model type and encoder choice.

The Big Picture

Model-based reinforcement learning (MBRL) aims to improve sample efficiency by training a world model—a learned simulator of the environment—and using it to plan or generate synthetic experience for policy learning. In the RL literature, Dreamer and its successors demonstrated that pixel-space world models trained with reconstruction objectives could dramatically outperform model-free methods on visual control tasks.

For vision-language agents, however, pixel-space world models introduce a double burden. First, they require a large image decoder to reconstruct future frames, which is computationally expensive. Second, the reconstruction objective encourages the world model to represent all visual details, including those that are irrelevant to the agent's task (e.g., the exact texture of a wall, the shadow of an object). This can distract representation learning from task- relevant features.

Dwell (Dreaming With Latent Embeddings for VLMs) proposes a minimal world model that operates entirely in latent space, predicting the next latent state rather than the next frame. The key design choice is to ground the latent space in a frozen pretrained visual encoder (V-JEPA2) rather than learning it jointly with the world model.

Why Existing Approaches Fall Short

Pixel-space Dreamer variants: These reconstruct future frames using a stochastic state space model and a convolutional decoder. For VLM agents, the decoder must handle high-resolution images aligned with the language context, which is expensive. The reconstruction loss does not distinguish task-relevant from task-irrelevant visual details.

End-to-end latent world models: Training the latent representation jointly with the world model is unstable for vision-language agents, where the latent space must simultaneously support language-conditioned reasoning and state prediction. Prior work (TD-MPC2 for VLMs) addresses this with complex auxiliary objectives; Dwell eliminates the problem by using a frozen encoder.

Model-free fine-tuning: Direct RL fine-tuning of VLMs without a world model is sample-inefficient and can catastrophically forget pretrained language abilities if the RL signal is too strong.

Core Mechanism

Frozen V-JEPA2 Encoder. All visual observations are encoded by a V-JEPA2 ViT-L encoder whose weights are frozen throughout training. V-JEPA2 was pretrained on video prediction; its embeddings are therefore already rich in temporal and semantic information, well-suited for predicting the next state.

Recurrent Latent Transition Model (RLTM). The core world model is a recurrent neural network: $h_{t+1} = \text{RLTM}(h_t, z_t, a_t, g_t)$ where h_t is the recurrent state, $z_t = \text{V-JEPA2}(o_t)$ is the current visual embedding, a_t is the action embedding, and g_t is the language goal embedding from the VLM.

Imagination and Policy Learning. For policy updates, the agent unrolls the RLTM for H steps in latent space: starting from a real observed state z_t, it generates an

imagined trajectory $(\hat{z}_{t+1}, \hat{z}_{t+2}, \ldots, \hat{z}_{t+H})$. A value head on top of the RLTM outputs reward predictions for each imagined state. The policy is updated by actor-critic on the imagined trajectories.

Technical Formulation

Let $z_t = f_{\text{JEPA}}(o_t) \in \mathbb{R}^d$ be the frozen V-JEPA2 embedding and $g = f_{\text{VLM}}(\ell)$ the language goal encoding. The RLTM is trained by minimizing:

$$\mathcal{L}_{\text{RLTM}} = \sum_{t=1}^{T} \|\hat{z}_t - z_t\|^2 + \lambda_r \sum_{t=1}^{T} (\hat{r}_t - r_t)^2$$

where $\hat{z}_t = h_t W_z$ and $\hat{r}_t = h_t w_r$ are predicted from the recurrent state.

The policy $\pi_\theta(a \mid h, g)$ and value $V_\phi(h)$ are trained with a model-based actor-critic objective over imagined trajectories of length H:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_{\hat{\tau}} \left[\sum_{k=0}^{H} \gamma^k \hat{r}_{t+k} + \lambda_e \mathcal{H}[\pi_\theta] \right]$$

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_{\hat{\tau}} \left[\left(V_\phi(h_{t+k}) - \hat{V}_{\text{target}} \right)^2 \right]$$

Learning or Inference Procedure

Warm-up: Collect N_{warmup} real environment transitions to initialize the replay buffer.

Alternate:

1. Collect M real steps with the current policy.
2. Update RLTM on the replay buffer for K gradient steps.
3. Unroll H-step imagined trajectories from the buffer.
4. Update π_θ and V_ϕ on imagined + real data.

The VLM backbone is fine-tuned with a small learning rate to adapt language-goal encoding to the new tasks, but the V-JEPA2 encoder is never updated.

What the Guarantee Says

No formal guarantee is claimed. The paper argues informally that using a pretrained visual encoder that already captures temporal dynamics (V-JEPA2) as the latent space provides a better-conditioned prediction target than a randomly initialized latent, reducing the risk of the world model diverging from the true dynamics. Empirically, the RLTM prediction error is lower for V-JEPA2 than for randomly initialized or reconstructed latents, supporting this claim.

Experimental Findings

- **Best overall:** Dwell achieves the highest mean task success across 5 visual environments among all trainable methods.
- **Compute reduction:** Imagination using the RLTM requires **2.3× less compute** than a pixel-space Dreamer baseline with the same planning horizon H.
- **Sparse reward advantage:** On tasks with sparse rewards, Dwell's margin over model-free baselines is 12.4 percentage points; on dense-reward tasks, the margin narrows to 3.1 points.
- **V-JEPA2 vs. other encoders:** Using CLIP instead of V-JEPA2 as the frozen encoder reduces performance by 8.2% on average, confirming the value of a temporally-aware encoder for the world model.

Ablations and Interpretation

Latent prediction vs. pixel reconstruction: Replacing RLTM with a pixel-decoder world model of similar capacity reduces performance by 4.1% and increases imagination compute by 2.3×, validating the latent-only design.

Frozen vs. fine-tuned V-JEPA2: Fine-tuning V-JEPA2 alongside the world model degrades performance by 5.3%, likely due to the joint instability of learning the latent space and predicting in it simultaneously.

Imagination horizon H: Performance peaks at $H = 5$ steps and degrades for $H > 10$, consistent with the compounding prediction error in long latent rollouts.

RLTM size: Scaling the RLTM from a 2-layer GRU to a 4-layer Transformer adds 2.7% success, suggesting that the world model's capacity is not yet a bottleneck at the evaluated environment scales.

Reference

Haoxi Li, Sikai Bai, Jianfei Ma, Qinglin Hou, Xiaocheng Lu, Tao Han, Jie Zhang, Song Guo. **Vision-Language Agents Can Learn in Their Own Latent Worlds.** arXiv:2610.36925. 2026. https://arxiv.org/abs/2610.36925

Keywords: multi-agent LLMs • collective reasoning • alignment measurement • information-efficient communication

Collective Emergent Alignment in Multi-Agent LLMs

REMAP Quantifies Interaction-Induced Reasoning Gains; Gap-Aware Factual Messages Amplify Them by +3.1 F1 While Cutting Token Cost by 58%

By Boxuan Wang, Tianxiang Sun, Linyang Li, Xipeng Qiu, Hang Yan

arXiv:2610.34836 • cs.AI • submitted 26 Oct 2026

Multi-agent LLM systems often outperform any individual agent on reasoning tasks—but is this due to genuine peer interaction or simply to extra self-refinement compute? This paper introduces Collective Emergent Alignment (CEA) to measure the interaction-specific gain, and the REMAP framework to quantify it at the level of individual facts. Across seven model families and two multi-hop benchmarks, CEA is consistently +6.3 F1 points above matched self-refinement, and gap-aware factual communication amplifies this by a further +3.1 points while reducing communication tokens by 58%.

AT A GLANCE

Problem: It is unclear whether multi-agent LLM performance gains reflect genuine collective intelligence or just additional self-refinement compute.

Why it matters: Multi-agent LLM systems are increasingly deployed in production; understanding whether interaction actually helps is essential for designing efficient systems.

Method: REMAP reference-entailment matrix measures per-fact alignment gains attributable to interaction; gap-aware factual messages route only missing facts between agents.

Result: CEA is +6.3 F1 over matched self-refinement; gap-aware communication adds +3.1 F1 and cuts tokens by 58%.

Evidence: 7 model families × 2 multi-hop QA benchmarks (MuSiQue, 2WikiMultiHopQA); group sizes 2, 4, 8.

The Big Picture

In a multi-agent LLM system, several models communicate over multiple rounds to jointly arrive at an answer. The intuition is that diverse agents will catch each other's errors and combine partial knowledge. Empirically, these systems often outperform single-model baselines. But this comparison is confounded: a multi-agent system at round k has performed more computation than a single agent that runs k self-refinement steps on its own. The improvement could be *collective*—arising from information transfer between agents— or *individual*—arising from additional self-refinement at matched compute.

This paper disentangles the two. It defines Collective Emergent Alignment (CEA) as the improvement in reference-grounded reasoning that arises specifically from inter-agent interaction, over and above what the same total compute would yield in independent self-refinement.

Why Existing Approaches Fall Short

Multi-agent vs. single-agent comparisons: Most prior work compares a multi-agent system directly to a single-agent baseline without controlling for compute. The multi-agent system always has more iterations and more tokens; any gain is ambiguous.

Aggregate metrics only: Existing evaluations report final F1 or accuracy but do not attribute performance gains to specific communication events. There is no way to tell which round or which message was responsible for a correct answer.

No formal model of collective gain: Prior work lacks a framework for defining what "collective" means in a measurable sense. Terms like "debate," "collaboration," and "consensus" are used loosely without formal grounding.

Core Mechanism

Reference-Entailment Matrix. REMAP constructs a binary matrix $M \in \{0,1\}^{A\times R\times F}$ where A is the number of agents, R the number of rounds, and F the number of atomic facts in the reference answer. $M_{a,r,f} = 1$ if fact f is entailed by agent a's response at round r. The matrix is populated using a pretrained natural-language inference model.

Collective Emergent Alignment. A fact f is attributed to CEA if:

- It first appears in some agent's response after a communication round (not at round 0), and
- It does not appear in the counterfactual trajectory where that agent self-refines without receiving peer messages.

The CEA score is the fraction of reference facts attributed to CEA under this definition.

Gap-Aware Factual Communication. Instead of exchanging full reasoning traces, each agent identifies which facts in its peer's last response are absent from or contradict its own response. It then routes a compact message containing only those "gap facts" to the peer. This is implemented with a lightweight LLM step that extracts

the gap-fact set from both responses.

Technical Formulation

Let $R_{a,r}$ denote agent a's response at round r, and let $E(R, f)$ be the entailment probability for fact f given response R. The binary matrix is:

$$M_{a,r,f} = \mathbf{1}[E(R_{a,r}, f) > \tau]$$

The Semantic Coverage of the collective at round r is:

$$\mathrm{SC}(r) = \frac{|\{f : \exists\, a,\ M_{a,r,f} = 1\}|}{|F|}$$

The CEA score is:

$$\mathrm{CEA} = \frac{1}{|F|} \sum_{f} \mathbf{1}[f \in F_{\text{collective}} \setminus F_{\text{individual}}]$$

where $F_{\text{collective}}$ are facts recovered by the multi-agent system and $F_{\text{individual}}$ are those recovered by the matched self-refinement baseline.

For gap-aware communication, the message from agent a to agent b at round r is:

$$\mathcal{G}^r_{a \to b} = \{f : M_{a,r,f} = 1,\ M_{b,r,f} = 0\}$$

and the message content is a natural-language summary of $\mathcal{G}^r_{a \to b}$ generated by a compact auxiliary LLM.

Learning or Inference Procedure

No model training is required for REMAP measurement. All LLMs are used in their pretrained, instruction-tuned states. The evaluation procedure is:

1. Initialize A agents with the same question.
2. Run R rounds of communication, alternating between standard full-trace and gap-aware factual modes.
3. Construct M using the NLI model after each round.
4. Compute SC, CEA, and per-round alignment gains.

The self-refinement counterfactual is obtained by re-running each agent independently for the same number of rounds, receiving no peer messages.

What the Guarantee Says

The paper provides no formal guarantee about CEA's optimality or about the gap-aware communication protocol. The theoretical contribution is the definition of CEA itself, which is grounded in the entailment matrix and admits a clean counterfactual interpretation. The gap-aware communication protocol is justified empirically: it consistently amplifies CEA and reduces token cost across all evaluated settings.

Experimental Findings

- **CEA is real:** Across 7 model families and both benchmarks, CEA averages +6.3 **F1 points** over matched self-refinement. All model families show positive CEA; none is negative.
- **Gap-aware communication:** Amplifies CEA by +3.1 **F1** while reducing communication tokens by **58%** compared to full-trace exchange.
- **Group size:** CEA increases with group size (2 to 8 agents) but with diminishing returns; going from 2 to 4 agents yields +4.1 F1, while going from 4 to 8 yields only +1.8 F1.
- **Benchmarks:** MuSiQue (harder, requiring 4-hop reasoning) shows higher CEA (+8.1) than 2WikiMultiHopQA (+4.5), consistent with more room for collective benefit when individual models struggle.

Ablations and Interpretation

Communication timing: Communicating every round is better than every 2 rounds (+1.2 F1), but the gap narrows with gap-aware messages (which are shorter and can be communicated more frequently).

Message content: Full-trace exchange vs. gap-aware facts: full-trace exchange achieves 3.1% lower F1 and costs 58% more tokens. Gap-aware messages are both cheaper and better.

NLI threshold τ: CEA is robust to $\tau \in [0.6, 0.8]$; below 0.6, noisy entailment decisions inflate CEA artificially; above 0.8, the definition becomes too strict and CEA underestimates the true gain.

Dominance phenomenon: In 22% of episodes, one agent provides >70% of the CEA facts. Sequential communication reduces this to 9%, suggesting that synchronous communication can lead to one agent dominating the group.

Reference

Boxuan Wang, Tianxiang Sun, Linyang Li, Xipeng Qiu, Hang Yan. **Collective Emergent Alignment: Measuring and Amplifying Interaction-Induced Reasoning Alignment in Multi-Agent LLMs.** arXiv:2610.34836. 2026. https://arxiv.org/abs/2610.34836